GUÍA
GIT Y GITHUB



Realizado por Sonia Fernández Pérez

ÍNDICE

GUIA RÁPIDA GIT 3
GUIA RÁPIDA DE GIT A GITHUB 3
ERROR DE SUBIDA DE ARCHIVOS +100MB 3
CONCEPTOS 4
1. Instalar y configurar Git (una sola vez) 4
2. Crear un repositorio con Git para clonar en github 4
Pasos 5
3. Crear y configurar la cuenta de GitHub 6
4. Clonar y actualizar repositorios desde Github a Git 7
Clonar (git clone <URL>) 7
Clonar un repositorio existente 7
Actualizar la copia local 8
Actualizar (git pull) 8
Crear tu propio repositorio 8
5. Ciclo de trabajo diario: add, commit, push 8
6. Archivos especiales 9
8. Ramas 10
Para qué sirven 10
Fusionar (merge) 10
9. Colaborar en proyectos ajenos (fork y pull request) 11
Conceptos 11
Quien contribuye 11
Quien es dueño del repositorio 11
10. Resumen de comandos 12
11. Hacer visible una página web en github 14

GUIA RÁPIDA GIT

1. Abrimos CMD en el repositorio local.
2. **Git add .** para añadir los cambios al área de preparación.
3. **git commit -m “descripción de los cambios”**
4. **git push origin master/main** (si has editado el nombre de la rama o no)
5. **git tag -a <nombre_del_tag> -m "Mensaje descriptivo"** para etiquetas
6. **git push --tags** para subir etiquetas actuales

GUIA RÁPIDA DE GIT A GITHUB

1. Cambiamos el nombre de la rama de git a main con **git branch -M main**.
2. **git remote add origin <url>** para vincular un repositorio remoto de github a git copiando la url en code.
3. **git push -u origin main** para subir el repositorio.

Nota: Si trabajas en equipo o desde otro ordenador, descarga y fusiona los últimos cambios de GitHub en tu equipo: **git pull origin main**.

ERROR DE SUBIDA DE ARCHIVOS +100MB

Para deshacer los commits locales manteniendo tus archivos intactos:

1. Borramos el commit hecho con el archivo mayor de 100mb, que al tener github límite de subida dará error. con: **git reset --soft origin/main**
2. Sacar el archivo pesado del área de preparación. **git rm --cached "ruta archivo"** . Este comando le dice a Git que deje de seguir el vídeo grande, pero no borrará el archivo de tu disco duro.
3. Usamos **add .** .Creamos el commit como siempre y subimos a github con **git push -u origin main**.

Ahora no dará error la subida ya que ningún archivo pesa más de lo máximo posible.

Conceptos:

- **Git** es un programa que se instala en tu ordenador y registra todos los cambios de tu proyecto. Sirve para volver atrás si algo sale mal, tener versiones y trabajar en varias líneas a la vez. Funciona con comandos en terminal.
- **GitHub** es una web que aloja proyectos controlados con Git. Sirve de copia de seguridad en internet y de escaparate de tu trabajo (muchas empresas buscan programadores allí). También permite colaborar con otras personas.
- **Repositorio**: la carpeta de un proyecto que Git vigila, con todo su historial. Existe una copia en GitHub (remoto) y otra en tu ordenador (local).

1. Instalar y configurar Git (una sola vez)

1. Instalar: **sudo apt install git** (en otros sistemas, desde git-scm.com/downloads).
2. Comprobar la instalación: **git --version**
3. Indicar tu identidad:

   git config --global user.name "Tu Nombre"

   git config --global user.email "tu@correo.com"
4. Opcional, para no reescribir la contraseña a cada rato:

   git config --global credential.helper 'cache --timeout=36000'

2. Crear un repositorio con Git para clonar en github

Hay dos formas de tener un repositorio:

- **Desde GitHub y luego clonarlo**. Se usa cuando empiezas el proyecto de cero en GitHub o quieres uno ajeno.
- **Desde Git, en tu ordenador.** Se usa cuando ya tienes una carpeta con código, o prefieres empezar en local y subirlo a GitHub después.

Crear un repositorio en Git significa decirle a Git que empiece a vigilar una carpeta. Hasta que lo haces, Git ignora esa carpeta y no registra nada. Al hacerlo se crea la carpeta oculta .git, donde Git guarda el historial, las versiones y la configuración. Un repositorio es, en esencia, tu carpeta más esa .git.

Pasos

1. Crea la carpeta del proyecto, o entra en una que ya tengas:

   ```
   mkdir mi-proyecto
   cd mi-proyecto
   ```

   Es importante estar dentro de la carpeta, porque Git actúa sobre el directorio en el que estás.

2. Inicializa el repositorio: **git init**

   Crea la carpeta .git y deja la carpeta lista para que Git registre cambios. No guarda nada todavía: solo empieza a vigilar. Puedes comprobarlo con **ls -a**, donde aparecerá .git.

   Si quieres que la rama principal se llame main (lo habitual hoy), usa **git branch -M main.**

3. Crea o copia tus archivos dentro de la carpeta.

4. Mira el estado: **git status**

   Mostrará tus archivos como "sin seguimiento" (untracked). Git los ve, pero aún no los registra.

5. Prepara los archivos (aperitivo): **git add .**

   Marca qué archivos van a entrar en el primer commit.

6. Haz el primer commit (comida):

   git commit -m "Primer commit"

   Guarda la primera versión en el historial. A partir de aquí ya puedes volver atrás, crear ramas, etiquetar versiones, etc.

7. Comprueba el historial: **git log**

Desde este punto el ciclo de trabajo es el de siempre: editar, add, commit. Todavía no hay push, porque no existe un repositorio remoto al que subir.

Después: conectarlo con GitHub

Cuando tengas la cuenta de GitHub, puedes subir este repositorio local:

1. En GitHub, crea un repositorio nuevo sin marcar Initialize this repository with README. Si lo marcas, GitHub crea un historial propio y chocará con el tuyo. (Es lo contrario de lo que hace el libro cuando se parte de GitHub.)
2. Copia la URL del repositorio.
3. Enlaza tu repositorio local con el de GitHub:

   git remote add origin <URL>

   origin es el nombre habitual del remoto. git remote -v muestra los enlaces configurados.

4. Sube tu historial:

   git push -u origin main

   -u recuerda la conexión, y a partir de entonces basta con git push y git pull.

Precauciones

- No ejecutes git init dentro de otro repositorio. Se crearía un repositorio anidado y daría problemas.
- No borres la carpeta .git a menos que quieras perder todo el historial. Tus archivos seguirían ahí, pero Git dejaría de conocerlos.
- Si ya tienes la carpeta con archivos, git init no los modifica ni los borra. Solo añade .git.

3. Crear y configurar la cuenta de GitHub

1. Entrar en github.com y pulsar Sign up.
2. Rellenar los datos: usuario (letras, números y guiones), correo y contraseña. Elegir el plan gratuito, que permite repositorios públicos ilimitados.

3. Completar la información adicional que pide. El perfil importa porque GitHub es tu carta de presentación como programador:

   Foto y nombre real: para que te reconozcan y se te tome en serio.

   Bio: tus estudios o cursos de informática, para dar contexto sobre ti.

   URL: tu web, blog o LinkedIn.

   Location (ciudad y país en inglés): las empresas buscan programadores filtrando por ciudad y lenguaje. Si no lo rellenas, no apareces en esos rankings.

   Available for hire: avisa de que estás disponible. Hay aplicaciones que rastrean perfiles buscando candidatos y miran esa casilla.

4. Configurar el perfil (avatar → Your profile → Edit profile): foto reconocible, nombre completo, Bio, URL (web o LinkedIn) y Location (ciudad y país en inglés). Marcar Available for hire si buscas trabajo.

4. Clonar y actualizar repositorios desde Github a Git

Clonar (`git clone <URL>`)

- **Qué hace**: copia un repositorio de GitHub a tu ordenador, con todo su historial.
- **Por qué clonar y no descargar los archivos**: una descarga es una foto fija. Si el repositorio cambia, tu copia queda desactualizada. Un clon mantiene la conexión con GitHub y se puede actualizar fácilmente.
- **Cómo**: copia la URL en GitHub (**Clone or download**), entra con `cd` en la carpeta donde lo quieres y ejecuta el comando.
- **Ventaja**: puedes experimentar sin miedo, porque tus cambios locales no afectan al repositorio original.

Clonar un repositorio existente

1. Copiar su URL en GitHub (botón Clone or download → icono del portapapeles).

2. Ir a la carpeta donde guardarlo: cd Documentos/Programacion/

3. Clonar: **git clone <URL>**

Actualizar la copia local

Actualizar (`git pull`)

> **Qué hace**: descarga a tu copia local los cambios que hayan aparecido en GitHub.
>
> **Por qué hace falta**: los clones **no se actualizan solos**, a diferencia de Dropbox o Google Drive. Hay que lanzar **git pull** a mano, dentro de la carpeta del proyecto.
>
> **Resultado**: *Already up-to-date* significa que no había nada nuevo. Si hubo cambios, Git lista los archivos modificados.

1. Entrar en la carpeta del proyecto: cd nombre-del-repo
2. Ejecutar **git pull**. Si ya está al día, responde Already up-to-date.

No se sincroniza solo: hay que ejecutar git pull a mano cada vez.

Crear tu propio repositorio

1. En GitHub, pulsar el + junto a tu avatar → New repository.
2. Poner un nombre claro, sin espacios ni caracteres especiales (por ejemplo hola-mundo-en-java) y una descripción.
3. Marcar Initialize this repository with README. Así el repositorio no queda vacío y se puede clonar.
4. Clonarlo en tu equipo con git clone <URL>.

5. Ciclo de trabajo diario: add, commit, push

La regla mnemotécnica del libro es "aperitivo, comida y postre".

1. Entrar en el repositorio y crear o editar archivos.
2. Mirar el estado cuando tengas dudas: git status
3. Add (aperitivo): **git add . --all**
4. Commit (comida): **git commit -m "Mensaje explicando los cambios"**

5. Push (postre): **git push**
6. Comprobar en GitHub que se ha actualizado.

Comandos útiles: **ls -a** (muestra también lo oculto, como .git), **tree** (vista en árbol), **cat archivo** (muestra su contenido).

6. Archivos especiales

README.md (lo primero que ve quien visita el repositorio). Está en formato Markdown:

- # título principal, ## apartado, ### subapartado
- *texto* cursiva y **texto** negrita
- Tres comillas invertidas con el lenguaje para bloques de código; para comandos se usa console
- [texto](URL) para enlaces
- Después de editarlo, repetir add, commit y push.

.gitignore (archivos que Git no debe seguir):

1. Crear el archivo: **gedit .gitignore**
2. Escribir un patrón por línea (por ejemplo *.bak y *.class). Las líneas con # son comentarios.
3. Hacer add, commit y push. Los archivos ignorados no se suben.

Historial de cambios: **git log** (o cat .git/logs/HEAD). Más opciones con git help log.7. Versiones (etiquetas)

Marcan un punto concreto del proyecto con un nombre, como `v1.0`. Se usan cuando completas un hito o terminas una funcionalidad. Así puedes volver a una versión estable y consultar o recuperar cómo estaba el proyecto en ese momento.

1. Etiquetar el estado actual: **git tag -a v1.0 -m 'Descripción de la versión'**
2. Listar etiquetas con descripción: **git tag -n**
3. Subir las etiquetas a GitHub: **git push --tags.** Un git push normal no las sube.

4. En GitHub se cambia de versión desde Branch: master → pestaña Tags.

8. Ramas

Una rama es una línea de trabajo independiente dentro del mismo proyecto. Al crear un repositorio hay una rama principal (`master`, hoy normalmente `main`).

Para qué sirven

- **Probar sin riesgo**: puedes experimentar con una idea nueva sin tocar el código estable de la rama principal.
- **Trabajar en paralelo**: varias personas pueden desarrollar funciones distintas a la vez sin pisarse.
- **Descartar o conservar**: si el experimento falla, abandonas la rama y la principal sigue intacta. Si funciona, lo incorporas.

1. Crear una rama y saltar a ella: **git checkout -b nombre-rama**
2. Trabajar normalmente (add y commit).
3. Primer push de la rama:

   git push --set-upstream origin nombre-rama. Los siguientes pushes son un simple git push.
4. Si el experimento funciona, volver a la rama principal: git checkout master

Fusionar (merge)

- **Qué es**: incorporar a una rama los cambios de otra.
- **Para qué**: cuando el experimento funciona, quieres que forme parte del proyecto principal. Sin fusionar, ese trabajo se queda aislado en su rama.
- **Pasos**:
  1. `git checkout master` te lleva a la rama que **recibirá** los cambios.
  2. `git merge "nombreRama"` trae a `master` los cambios de la otra rama.
  3. `git push` sube la fusión a GitHub.

- **Resultado**: `master` tiene ahora los cambios de la otra rama. La rama experimental sigue existiendo, pero ya no hace falta, y puedes continuar el trabajo en la principal.

9. Colaborar en proyectos ajenos (fork y pull request)

Un repositorio público puede ser visto por miles de personas. Si quieres arreglar un fallo o mejorar un proyecto ajeno, **no puedes modificarlo directamente**, porque no es tuyo. Este proceso permite proponer cambios y que el dueño decida si los acepta.

Conceptos

- **Fork**: una copia del repositorio ajeno en tu propia cuenta de GitHub. Es tuya, así que puedes modificarla libremente sin afectar al original.
- **Pull request**: una petición al dueño del original para que incorpore tus cambios. Él los revisa antes de aceptarlos, lo que protege el proyecto de cambios no deseados.

Quien contribuye

1. En el repositorio original, pulsar Fork. Se crea una copia en tu cuenta.
2. Clonar tu copia: **git clone <URL-de-tu-fork>**
3. Hacer los cambios y ejecutar **git add . --all**, **git commit -m** "..." y **git push.**
4. Si editas algo directamente en GitHub (icono del lápiz, Edit this file, luego Commit changes), ejecuta después git pull para actualizar tu copia local.
5. En GitHub, pulsar Create **pull request**, escribir título y descripción, y pulsar de nuevo Create pull request. Quedará como Open.

Quien es dueño del repositorio

1. Entrar en la pestaña Pull requests.
2. Abrir la solicitud y revisar los comentarios, los commits y los archivos modificados.

3. Pulsar Merge pull request y luego Confirm merge, con un comentario. La solicitud pasa a Merged.

10. Resumen de comandos

Comando	Para qué sirve
git init	Convertir la carpeta actual en un repositorio Git
git init -b main	Lo mismo, llamando main a la rama principal
git remote add origin <URL>	Enlazar el repositorio local con uno de GitHub
git remote -v	Ver los repositorios remotos enlazados
git push -u origin main	Primera subida a GitHub, recordando la conexión
git clone <URL>	Copiar un repositorio de GitHub a tu equipo
git pull	Traer a tu copia local los cambios de GitHub
git status	Ver el estado y qué hacer a continuación
git add . --all	Preparar todos los cambios para el commit
git commit -m "..."	Guardar los cambios en el historial local
git push	Subir los commits a GitHub
git log	Ver el historial de commits
git tag -a v1.0 -m "..."	Marcar una versión

git push --tags	Subir las etiquetas a GitHub
git checkout -b rama	Crear una rama y cambiar a ella
git checkout rama	Cambiar de rama
git merge rama	Traer a la rama actual los cambios de otra

11. Hacer visible una página web en github

Para hacer visible una página web en github:

1. debemos primero comprobar que el archivo index.html esté en la raíz principal del proyecto.
2. Después estando en la raíz principal, nos vamos a settings, a la pestaña pages, ponemos:
    a. En Build and deployment “Deploy from a branch.
    b. En Branch elegimos main y raíz /(root).
3. Pulsamos aceptar.

Si esperamos unos minutos, nos saldrá el enlace a nuestra web.

